ENTERPRISE DATA ENGINEERING

NYC Mobility Analytics Platform

➔ Objective: End-to-End Enterprise Data Solution for NYC High-Volume For-Hire Vehicle (HVFHV) Trip Data

CORE STACK

DuckDB ➔ **PostgreSQL 16** ➔ **Docker** ➔ **Apache Superset**

~240M records ➔ Full-year 2025

~1.5 GB/month raw Parquet

Business Context & Technical Challenges

➔ Analyze demand trends, vendor revenue split, and spatial patterns across NYC.

KEY CHALLENGES

1

Local Hardware Constraints

➔ Heavy I/O bottlenecks in traditional RDBMS ingestion.

2

Schema Drift

➔ Schema variations in historical datasets (e.g., missing fee fields).

3

Data Quality

➔ Anomalies, zero-value distances, and timestamp mismatches in raw data.

Hybrid Lakehouse Architecture

Architecture Pattern: Medallion Lakehouse Architecture

BRONZE

Data Lake

Raw Parquet files stored immutably on disk.

SILVER

Compute

DuckDB in-memory engine ➔ Blazing-fast filtering & transformations.

GOLD

Data Warehouse

PostgreSQL 16 ➔ Optimized Star Schema for analytics.

Flow: Parquet (Bronze) ➔ DuckDB SQL (Silver Clean/Quarantine) ➔ PostgreSQL (Gold Data Warehouse)

Data Quality Framework & Fail-Fast Mechanism

12

Strict Data Quality Rules (R01 to R12)

Source Rows



Valid Rows ➔ trips_clean

Invalid Rows ➔ trips_quarantine (full audit trail)

Reconciliation Check: Total Source Rows = Clean Rows + Quarantined Rows.

Resilience: Fail-Fast execution halts pipeline immediately on reconciliation mismatch.

Schema Evolution & Resilience Testing

Drift Challenge: Historical files (e.g., 2024 data) lack newer TLC fee columns (e.g., cbd_congestion_fee).

HANDLING STRATEGY

1 Detect

➔ Programmatic column detection in DuckDB.

2 Inject

➔ Dynamic Schema Evolution
➔ Auto-inject missing columns with COALESCE(..., 0).

3 Result

➔ Backfill support across historical years without breaking pipeline schema contracts.

Dimensional Data Modeling (Gold Layer)

dim_time

Hourly granular mapping

dim_zone

Role-playing dimension for Pickup & Dropoff locations

dim_company

HVFHV Vendors: Uber, Lyft, Via

Schema Type: Star Schema (Dimensional Modeling).

fact_trips

Grain: 1 row per completed trip, ~240M rows

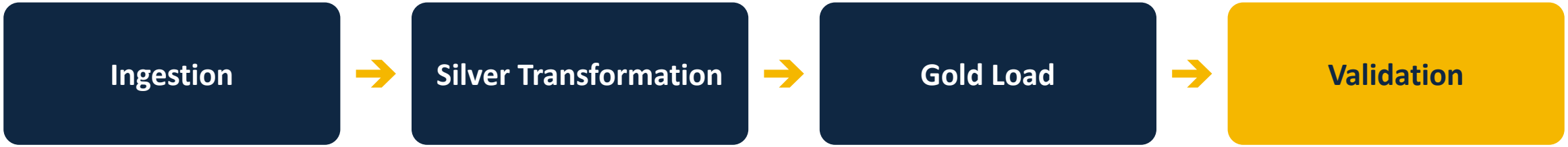
Integrity: Unknown/Missing attributes mapped to surrogate Key 0.

Fact Table ➔ Dimension Tables

Production Orchestration & Reliability

Orchestrator: Custom Production Scripting Orchestrator (run_pipeline_v2.bat / run_whole_2025.bat)

SEQUENTIAL EXECUTION



Idempotency

➔ DELETE before INSERT parameterization by source_file.

Operational Guardrails

➔ Automated retry limits, structured logging in /logs, environment cleanup.

Performance Engineering & Optimization

Optimization 1: B-Tree Indexing

➔ Applied on time_key and pu_zone_key in fact_trips.

83%

Reduced query time

Before: 3.1 minutes



After: 32 seconds



Optimization 2: Pre-Aggregation

➔ Built summary table agg_hourly_zone for BI dashboard consumption.

99.3%

Reduced dashboard load time

Before: 1.7 minutes

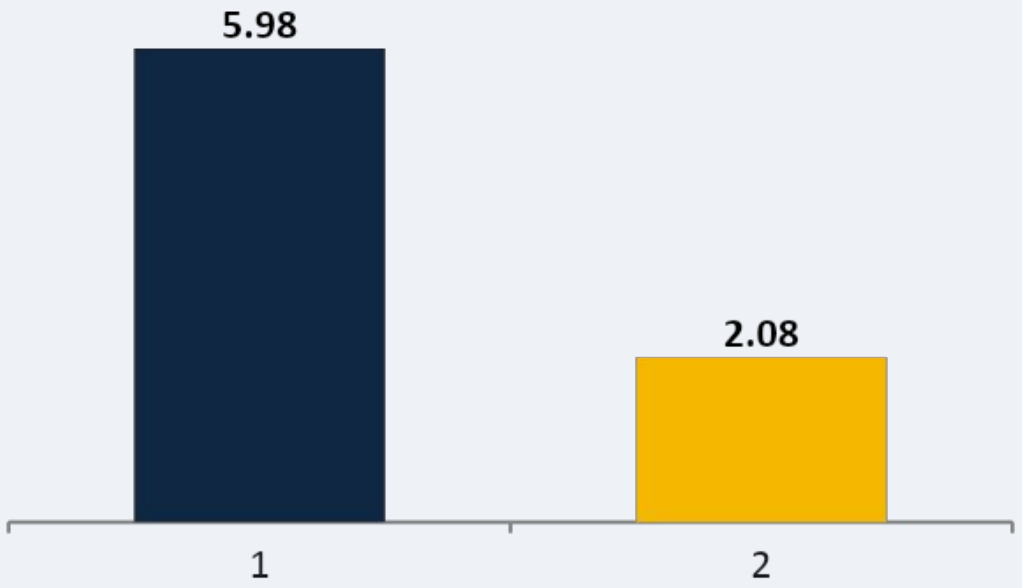


After: < 1 second



Business Insights & SQL Analytics

Market Split: Revenue ($B)



Uber leads market share with over $5.98B in revenue (174M trips), followed by Lyft ($2.08B).

1

Demand Peak

➔ Hourly demand peaks during evening rush hours (5:00 PM – 7:00 PM).

2

Weekend Shift

➔ Friday & Saturday generate peak trip volume; Monday exhibits the lowest demand.

3

Airport Demand

➔ JFK & LaGuardia dominate high-value pickup zones, peaking early morning and late afternoon.

Executive Summary & Dashboard

BI Layer: Apache Superset connected to PostgreSQL Gold Layer.

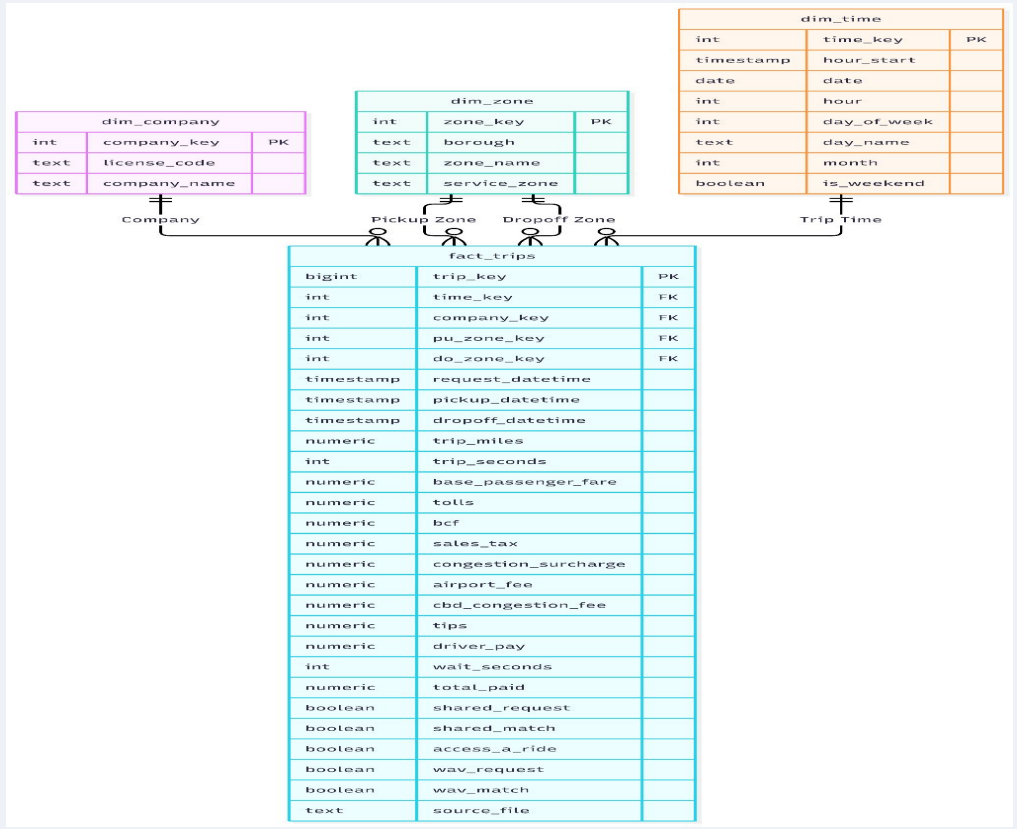
Total Trips	Total Revenue	Average Fare	Demand Heatmaps

SUMMARY DELIVERABLES

- ➔ **Scalable Hybrid Lakehouse Architecture.**
- ➔ **Automated Data Quality & Schema Evolution.**
- ➔ **Sub-second BI Query Performance (< 1s execution).**

Repository: Fully documented GitHub source code and batch pipelines.

Star Schema Design

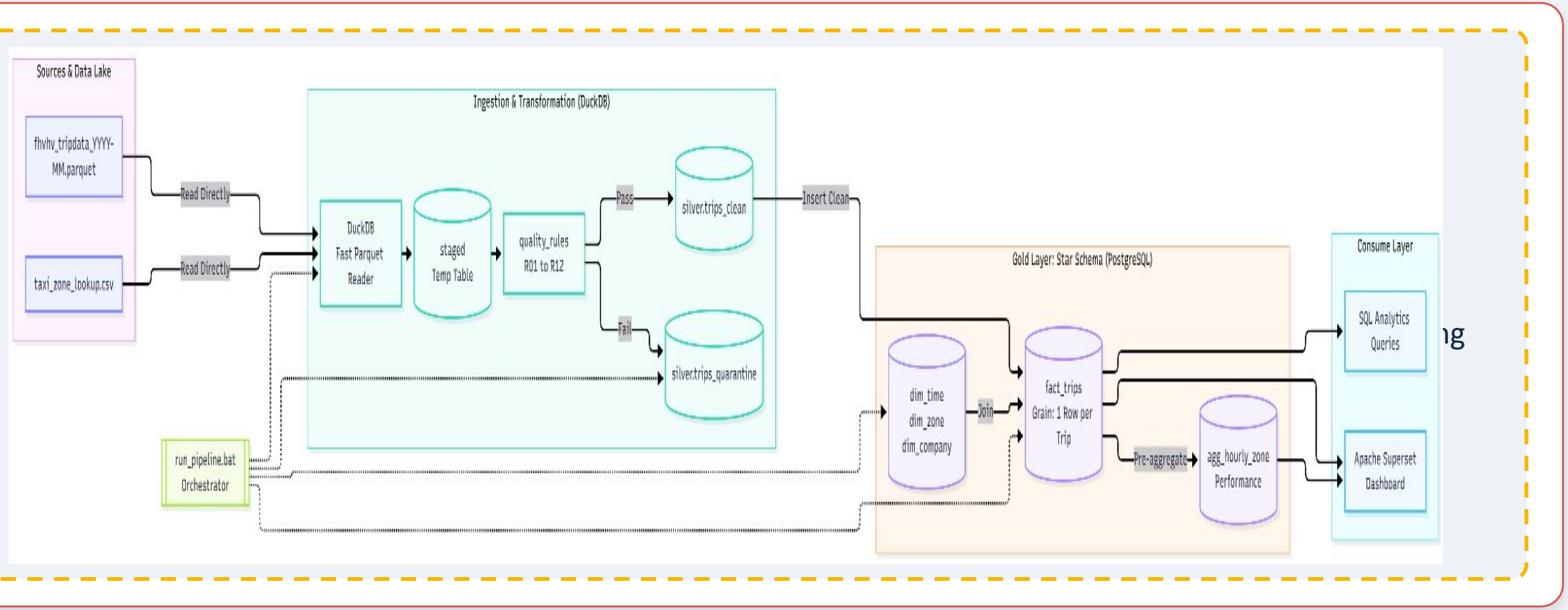


One fact table, three dimensions

- Simple joins
- Fast aggregations
- BI-ready

Fact_trips · Dim_time
Dim_zone · Dim_company

Lakehouse: Bronze → Silver → Gold



Bronze: raw Parquet **Silver: DuckDB transform** **Gold: PostgreSQL star schema**

Right tool for each layer: DuckDB for compute, PostgreSQL for serving